

프로젝트명	GMP 시스템 Spring Boot 3.x 전환 및 현대화 프로젝트		
연계/소속	S*운영팀 / 포커스원	개발 인원	1명 (독립 수행)
주요 업무	백엔드 개발 및 시스템 마이그레이션 전담		
담당 역할	프레임워크 마이그레이션 총괄, 멀티모듈 아키텍처 재설계, 보안 취약점 능동 발굴·수정, 인증 시스템 구축		
기술 스택	Java 17, Spring Boot 3.3.2, Spring Security 6.x, MyBatis, JPA/Hibernate, MariaDB, Gradle 8.8, Git		
업무 기간	2025.11 ~ 현재 진행중 (약 5개월)		

10년 이상 운영된 레거시 시스템 (Java 1.7 / Spring Framework 3.x)을 Java 17 / Spring Boot 3.3.2 기반으로 1인 독립 전환하며, 단순 이식이 아닌 **아키텍처 재설계** · **보안 강화** · **코드 품질 개선**을 병행했습니다.

1. 마이그레이션 전략 수립 — 순서가 틀리면 원인을 찾을 수 없다

PROBLEM

Jakarta EE 전환, Spring Security 재작성, MyBatis 설정 교체, JSP 뷰 연동까지 동시에 손을 대면 어디서 오류가 났는지 추적이 불가능해집니다. 특히 코드 수정 후 폐쇄망 서버에 배포해야만 확인이 가능한 환경이었기 때문에, 원인 불명의 오류 하나가 수 시간의 낭비로 이어지는 구조였습니다.

ACTION

처음에는 문서에 나온 마이그레이션 가이드 순서대로 따라가려 했습니다. 그런데 실제로 시작해보니 여러 계층이 한꺼번에 깨지면 어디가 원인인지 알 수 없다는 걸 초반에 바로 체감했습니다.

"빌드가 되는 상태를 절대 깨지 않는다"는 원칙을 세우고, **인프라(빌드·DB 연결) → 보안·인증 → 비즈니스 로직 → 뷰** 순으로 계층을 나눠 단계적으로 진행했습니다. 디버깅의 기본인 "변수를 하나씩 제거한다"는 원리를 마이그레이션 프로세스에 그대로 적용한 것입니다.

폐쇄망 제약을 고려해 로깅 인프라를 가장 먼저 구축했습니다. ViewResolver TRACE · Spring Security DEBUG 수준의 로그 레벨을 설정하고, 클라이언트 IP와 세션 ID를 로그에 자동 포함하는 **커스텀 Logback Converter**를 직접 구현했습니다. 각 단계마다 TestController와 계층별 독립 테스트 엔드포인트를 구성해 해당 계층만 검증한 뒤 다음으로 이동했습니다.

RESULT

전체 **18단계 마이그레이션**을 원인 불명 오류로 인한 중단 없이 완료했습니다. 로그 기반 디버깅 환경 선구축으로 문제 발생 시 원인 파악 시간을 크게 단축했고, 버그 수정 주기도 **3일 → 1일**로 줄었습니다.

2. 빌드 환경 재설계 — 폐쇄망에서 빌드가 안 된다

PROBLEM

기존 Maven 빌드 시스템은 폐쇄망 환경에서 외부 의존성을 받아올 수 없어 빌드 자체가 불가능했습니다. 공통 라이브러리 모듈도 2개(gmp-framework, gmp-common)로 분산되어 있어 어느 모듈에 무엇이 있는지 파악하는 것부터 시간이 걸렸습니다.

ACTION

빌드 시스템을 Gradle로 전환하면서 가장 먼저 고민한 것은 폐쇄망 대응이었습니다. Gradle은 로컬 파일 시스템을 Maven 저장소처럼 사용할 수 있어, **gradle-lib/ 폴더에 의존성 JAR를 직접 저장하고 오프라인으로 빌드**하는 구조를 만들었습니다.

모듈 통합은 선택의 문제였습니다. 2개를 유지하면 기존 구조를 건드리지 않아도 되지만, 실제로 두 모듈의 역할이 명확히 구분되지 않아 어디에 코드를 추가해야 할지 매번 판단이 필요했습니다. 유지보수 관점에서 **gmp-framework 단일 모듈로 통합**하고, 공통 의존성을 root build.gradle로 일원화했습니다.

RESULT

- 폐쇄망 환경에서 오프라인 빌드 가능 구조 완성
- Maven 대비 전체 빌드 시간 단축 (3분 42초 → 2분 40초, 28% 감소)
- 모듈 통합으로 신규 코드 추가 위치 판단 비용 제거

3. 코드를 읽다가 발견한 보안 취약점 — 기능 테스트로는 안 잡힌다

PROBLEM

레거시 로직을 Spring Boot 방식으로 이식하는 과정에서 기능이 동작하는지 확인하는 것만으로는 놓치는 문제들이 있었습니다. 코드를 한 줄씩 읽으며 옮기는 과정에서 보안상 위험한 로직들이 눈에 들어오기 시작했습니다.

ACTION

첫째, 파일 다운로드 엔드포인트에 파일 경로 검증이 전혀 없었습니다. 파라미터에 "../"를 넣으면 서버 내 임의 경로의 파일에 접근 가능한 구조였습니다. "내부망이니까 괜찮겠지"라고 넘어갈 수 있었지만, 내부 사용자의 실수나 의도치 않은 접근도 막아야 한다고 판단해 파일명에 ".", "/", "\" 포함 여부를 검증하는 로직을 추가했습니다.

둘째, 로그인 차단 로직에서 isBlock()이 true를 반환해도 이후 로그인 코드가 그대로 실행되는 구조였습니다. 차단 판단 후 즉시 return이 빠져 있어, 차단된 사용자가 실제로는 로그인이 되는 버그였습니다. 함께 검토하다 인증 SMS 검증 메서드에서 **AND 연산자를 써야 할 자리에 OR가 쓰인 논리 버그**도 발견해 함께 수정했습니다.

이 경험에서 기능을 만드는 것과 코드를 읽는 것은 다른 행위라는 걸 체감했습니다. 코드를 옮기는 작업이 동시에 코드 리뷰가 되었고, 그 과정에서 테스트로는 발견하기 어려운 버그들이 드러났습니다.

RESULT

- Path Traversal 취약점, 인증 우회 버그, 논리 연산자 오류 등 운영 배포 전 선제 발굴 및 수정 완료
- Jasypst 암호화를 적용해 DB 접속 정보도 설정 파일에 평문으로 노출되지 않도록 처리

4. 데이터 접근 설계와 코드 구조 개선 — 일관성이나 효율이나

PROBLEM

로그인 실패 이력 관리 기능을 신규 설계하면서 기존 MyBatis로 통일할지, JPA를 도입할지 결정해야 했습니다. 동시에 레거시 코드를 이식하면서 Controller에 비즈니스 로직이 섞여 있고, 필드 주입 방식의 DI가 혼재하는 구조 문제도 정리가 필요했습니다.

ACTION

MyBatis vs JPA 결정에서 처음에는 기존 스택과의 일관성을 위해 MyBatis 통일이 맞다고 생각했습니다. 그런데 로그인 이력 도메인은 단순 저장과 집계 조회가 대부분이라, MyBatis XML을 새로 작성하는 것보다 **Spring Data JPA의 Repository 추상화**를 쓰는 게 훨씬 간결하고 타입 안전한 구조였습니다.

결론은 **"복잡한 동적 쿼리는 MyBatis, 단순 CRUD 신규 도메인은 JPA"**로 용도에 따라 분리하는 것이었습니다. 나중에 팀원이 합류하면 두 방식이 섞인 이유를 설명해야 한다는 점을 트레이드오프로 인식하고 결정 이유를 문서로 남겼습니다.

코드 구조는 필드 주입(@Autowired)을 **생성자 주입(@RequiredArgsConstructor + final)**으로 전환하는 작업을 병행했습니다. 처음에는 큰 차이가 없다고 느꼈는데, 실제로 전환하면서 CompanyService ↔ UserExService 간 순환 참조가 빌드 시점에 바로 드러났습니다. 필드 주입이 숨기고 있던 설계 문제가 생성자 주입으로 바꾸자 즉시 가시화된 것입니다. 이 경험으로 DI 방식이 단순한 스타일 차이가 아니라 **설계 문제를 드러내는 도구**가 된다는 걸 이해하게 됐습니다.

RESULT

- MyBatis + JPA 하이브리드 전략으로 레거시 쿼리 자산 유지와 신규 도메인 개발 효율 양립
- 생성자 주입 전환으로 순환 참조 구조 발굴 및 해소
- Controller · Service 계층 책임 분리 완료

핵심 성과

✔ 대규모 마이그레이션 1인 완료

- Java 1.7 / Spring 3.x → Java 17 / Spring Boot 3.3.2
- 단계적 전략으로 18단계 전 구간 독립 수행 완료

✔ 보안 취약점 선제 발굴·수정

- Path Traversal, 인증 우회 버그, 논리 연산자 오류 능동 탐지
- 운영 배포 전 전수 해결 완료

✔ 폐쇄망 맞춤 빌드·디버깅 환경 구축

- 오프라인 Gradle 빌드 구성 및 계층별 로깅 전략 수립
- 테스트 불가 환경에서도 빠른 원인 파악 구조 완성

✔ 기술 선택의 트레이드오프 인식

- MyBatis + JPA 하이브리드 도입, 생성자 주입 전환
- 설계 결정의 이유와 한계를 함께 문서화